

# Команды для работы с модулями ядра

Владислав Богданов

2026-06-16

# Оглавление

|                                           |   |
|-------------------------------------------|---|
| 1. Полезные команды .....                 | 1 |
| 1.1. <code>insmod</code> .....            | 1 |
| 1.2. <code>rmmod</code> .....             | 1 |
| 1.3. <code>lsmod</code> .....             | 1 |
| 1.4. <code>depmod</code> .....            | 1 |
| 1.5. <code>modprobe</code> .....          | 1 |
| 1.6. <code>modinfo</code> .....           | 1 |
| 1.7. <code>cat /proc/devices</code> ..... | 2 |
| 1.8. <code>udevadm</code> .....           | 2 |
| 1.9. <code>kmalloc</code> .....           | 2 |
| 1.10. <code>vmalloc</code> .....          | 3 |
| 1.11. <code>mmap</code> .....             | 3 |

# 1. Полезные команды

## 1.1. insmod

Назначение: загрузка модуля по горячему

## 1.2. rmmod

Назначение: убрать из системы

## 1.3. lsmod

Назначение: список устройств в системе

## 1.4. depmod

Назначение: устанавливает зависимости модулей в системе в папке */lib/modules/7.0.0-22-generic* расположены модули загружаемые системой, создаем там каталог *misc* в него копируем свои модули, после запуска *depmod* он перестраивает зависимости и данные модули прекрасно запускаются с помощью *modprobe*. *depmod* нужно вызывать после того, как модули скопированы в системную папку модулей (например, *lib/modules/misc*).

## 1.5. modprobe

Назначение: позволяет мягким образом работать с модулями. Загружает/Выгружает модули с зависимостями.

```
//load module
sudo modprobe hello
//delete module
sudo modprobe -rf hello
```

## 1.6. modinfo

Назначение: показывает информацию о модуле

```
modinfo param_demo1.ko
filename:      /home/user/prj/linux/param_demo1/param_demo1.ko
author:        user
license:       GPL
srcversion:    23D76AE67C80B7BE8692A2F
depends:
name:          param_demo1
retpoline:     Y
```

```
vermagic:      7.0.0-22-generic SMP preempt mod_unload modversions
parm:         m_count:module_counter (int)
parm:         m_char:module_string (charp)
```

## 1.7. cat /proc/devices

Назначение: можно посмотреть старшие номера устройств

## 1.8. udevadm

Назначение: показывает полное дерево путей (DEVPATH).

```
# Смотрим физический порт (пример для ttyS0 или ttyAMA0)
udevadm info -q path -n /dev/ttyS0
# Вывод: /devices/platform/soc/serial@.../tty/ttyS0

# Смотрим виртуальный терминал
udevadm info -q path -n /dev/tty1
# Вывод: /devices/virtual/tty/tty1
```

## 1.9. kmalloc

Выделение памяти в ядре

параметры (объем, флаги)

Флаги:

1. GFP\_KERNEL - означает что память выделяется в нижней памяти от имени модуля, срабатывает от имени модуля и делает системный вызов выделения памяти. В случае недостатка памяти данный флаг разрешает наш процесс приостановить пока не наберется свободной памяти. То есть флаг блокирующий. Ядро при этом будет пытаться максимально быстро искать способы выделить память. В частности будет использоваться своп для пользовательских процессов. Флаг встречается в 90% программ. **Тонкий момент** флаг не всегда хорош. Не годится тогда когда процесс ядра не может спать. К примеру: обработчики прерываний, таймеры ядра.
2. GFP\_ATOMIC - ядро выделяет память моментально или выдает сбой при невозможности. В ядре всегда есть некий запас страниц памяти для надежной работы. (ЛОГИКУ ЛУЧШЕ ПРИВЯЗЫВАТЬ К СТРАНИЦАМ). Просто так использовать данный флаг не рекомендуется. Вообще получить сбой при выделении памяти очень сложно и сбой обычно означает либо аппаратные проблемы либо проблемы в логике программы.
3. GFP\_USER - выделение памяти под страницы пространства пользователя, то есть память не под себя а под пользовательский процесс. При этом память выделяется в нижней памяти.
4. GFP\_HIGH\_USER - - // - // - выделяет память в верхней памяти для пространства

пользователя.

Вспомогательные флаги используемые совместно с выше перечисленными:

1. GFP\_NOIO - при выделении памяти запрещены любые операции ввода вывода.
2. GFP\_NOFS - при выделении памяти запрещены любые операции с файловыми системами.

и тд.

Так же есть специальный флаг **\_\_GFP\_DMA** - заставляет выделить память в специальной DMA совместимой памяти. Она находится в диапазоне приверегированных адресов, там где периферийные устройства выполняют так называемый dma обращения.

Ядро управляет памятью системы таким образом, что память выделяется так называемыми кусками (slab) размером в страницу. Поэтому ядро и модули ядра могут выделять лишь некоторые заранее определенные объемы памяти с фиксированным количеством байт, то есть по сути кратно странице.

linux/slab.h - библиотека управления памятью.

## 1.10. **vmalloc**

Выделяет непрерывную с виртуальной точки зрения область памяти, соответственно можно не пользоваться vmalloc, а использовать alloc\_page. vmalloc работает через alloc\_page. vmalloc - основной инструмент выделения памяти в linux. **В ядре использовать не рекомендуется** он может быть фрагментирована, работает медленнее и из-за того что эта память в некоторых архитектурах может быть расположена строго на одном каком то участке памяти на нее может быть выделено места меньше памяти чем на нижнюю, зависит от архитектуры. Может потребоваться для хранения большого количества информации.

## 1.11. **mmap**